

Fast Virtual Gate Extraction For Silicon Quantum Dot Devices

Shize Che, Seongwoo Oh, Haoyun Qin, Yuhao Liu, Anthony Sigillito, Gushu Li

{shizeche,ohseong,qhy,liuyuhao,asigilli,gushuli}@seas.upenn.edu

University of Pennsylvania, USA

ABSTRACT

Silicon quantum dot devices stand as promising candidates for large-scale quantum computing due to their extended coherence times, compact size, and recent experimental demonstrations of sizable qubit arrays. Despite the great potential, controlling these arrays remains a significant challenge. This paper introduces a new virtual gate extraction method to quickly establish orthogonal control on the potentials for individual quantum dots. Leveraging insights from the device physics, the proposed approach significantly reduces the experimental overhead by focusing on crucial regions around charge state transition. Furthermore, by employing an efficient voltage sweeping method, we can efficiently pinpoint these charge state transition lines and filter out erroneous points. Experimental evaluation using real quantum dot chip datasets demonstrates a substantial $5.84\times$ to $19.34\times$ speedup over conventional methods, thereby showcasing promising prospects for accelerating the scaling of silicon spin qubit devices.

CCS CONCEPTS

• **Computer systems organization** → **Quantum computing**; • **Hardware** → *Electronic design automation*.

KEYWORDS

Quantum Dot, Device Tuning, Virtual Gate

ACM Reference Format:

Shize Che, Seongwoo Oh, Haoyun Qin, Yuhao Liu, Anthony Sigillito, Gushu Li. 2024. Fast Virtual Gate Extraction For Silicon Quantum Dot Devices. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3655923>

1 INTRODUCTION

Electron spin qubits in gate-defined silicon quantum dots are one of the most promising candidates to implement large-scale quantum computers due to long coherence times [23, 29], small size [4], and fast single- and two-qubit operations [15, 17, 28]. In addition, silicon quantum dots can be fabricated *en masse* using the existing semiconductor fabrication infrastructure, which offers enormous potential for rapid scaling [34]. As of now, most of the DiVincenzo's criteria [5] for a quantum computer have been satisfied for silicon spin qubits: long coherence times [22, 29], state preparation [8, 15,

27], high-fidelity universal gates [9, 15, 19, 27, 31], and high-fidelity readout [7, 14, 21]. Recently, the quantum dot chips of 12 qubits and 16 qubits have been experimentally demonstrated by Intel [10] and Delft [2], respectively.

While fabricating large-scale quantum dot arrays is relatively straightforward, controlling such large-scale arrays is a major hurdle. Quantum dot device tuning is a multi-stage process in which the voltages applied to metallic electrodes are varied until the device supports individual electrons beneath specific gates. Depending on the qubit encoding, the ideal electronic charge configuration can vary. This process becomes increasingly complicated and time-consuming as the number of qubits increases. Currently, it can take hours of human effort to achieve a satisfying voltage configuration. This has motivated the development of computer-aided tuning techniques targeting various stages of the tuning process [12, 16, 18, 24, 32, 33].

This paper focuses on a critical step in the tuning process, establishing independent control over the energy potential of each dot in a device. Ideally, a gate voltage only affects the potential level of its corresponding dot, but due to cross-capacitance, it also affects the potential levels of nearby dots. A widely adopted technique for establishing such “one-to-one” control is constructing the so-called virtual gate with a linear combination of the capacitively coupled gates to a given dot [1]. This virtual gate technique has been successfully applied to quantum dot array devices [13, 26].

Existing automation techniques for virtual gate extraction rely on analyzing complete *charge stability diagrams*, deploying computer vision techniques such as the Hough transform [12, 18], or more advanced techniques such as convolutional neural network [32]. However, the techniques' reliance on low-noise charge stability diagrams dramatically limits their scalability since obtaining a charge stability diagram on an actual device often takes minutes. The number of diagrams scales linearly with the number of dots, creating multi-hour overhead on devices with merely tens of qubits. As the number of qubits grows, a fast automated procedure for virtual gate extraction will be vital for scaling up silicon spin qubit devices.

In this paper, we make a key observation that there is a large redundancy in existing virtual gate extraction methods. Most of these data points in the charge stability diagram are not necessary, and only those data points on or close to the charge state transition lines contribute to constructing the virtualization matrix. Suppose we can locate the region of transition lines. In that case, we can probe only the points near them to significantly reduce the required experimental data and thus accelerate the virtual gate extraction.

To this end, this paper proposes a fast virtual gate extraction method. By making reasonable assumptions based on the device physics, our method can locate the transition lines and construct the virtualization matrices with much fewer experimental requirements. **First**, the device physics puts constraints on the possible values of the transition line slopes. Thus, the transition lines can only appear

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
DAC'24, San Francisco, CA, USA,

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0601-1/24/06
<https://doi.org/10.1145/3649329.3655923>

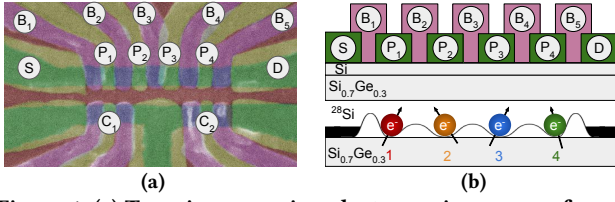


Figure 1: (a) Top-view scanning electron microscopy of a quadruple-dot device, (b) Lateral-view of the device's dot-side

on a specific region in the entire charge stability diagram. By quickly locating this specific region, we can greatly reduce the number of voltage configurations we need to experimentally test. **Second**, we further propose an efficient voltage configuration sweeping method to adaptively narrow our data probing to the critical area around the transition lines to further reduce the experimental requirements.

We evaluate the proposed fast virtual gate method on charge state datasets of real quantum dot chips from the qflow dataset [35]. Experimental results show that our method can outperform the baseline method using Hough transform with about $5.84\times$ to $19.34\times$ speedup with an even higher success rate. The speedup mostly comes from the reduction in the number of data points probed as our method only requires about 10% of the entire CSD data to locate the transition lines.

2 BACKGROUND

In this section, we introduce the basics of quantum dot technology and gate voltage tuning. Note that the term ‘gate’ in the rest of this paper refers to the hardware gate electrode fabricated on the chip rather than the unitary transformations in the quantum algorithm/software.

2.1 Quantum Dot

The quantum dot devices under consideration in this paper have the structure shown in Figure 1 (a). On one side of this device (the upper half of Figure 1 (a)), it consists of a source (marked by S), a drain (marked by D), a set of plunger gates (marked by $P_1 \sim P_4$) and a set of barrier gates (marked by $B_1 \sim B_5$). By adjusting the gate voltages, one can trap electrons under the gates, forming the so-called ‘dots’. On the opposite side (the lower half of Figure 1 (a)) of the device, there are two charge sensors (C_1 and C_2), both of which are single quantum dots. The charge sensors’ conductance is sensitive to changes in the local electrostatic potential. It can, therefore, detect changes in the number of electrons in the dots through current change in the sensors.

The upper half of Figure 1 (b) shows the cross-sectional view of the plunger ($P_1 \sim P_4$) and barrier gates ($B_1 \sim B_5$) as well as the additional layers and the trapped electrons below the gates (charge sensors are on the opposite side and not shown). The device has multiple layers made of different semiconductor materials. The electrons are sandwiched in the middle layer. They are subject to the electrostatic potential (represented by the curvy line in the lower half of Figure 1 (b)) generated from tuning the gate voltages. A desirable potential profile, like the one in Figure 1 (b), creates four dots labeled from 1 to 4 under each plunger gate. In Figure 1 (b), each dot confines and stabilizes one electron, forming the Loss-DiVincenzo qubits [11], where $|0\rangle$ and $|1\rangle$ are encoded as the spin up and down of the electron, respectively.

2.2 Charge Stability Diagram

To understand how gate voltage tuning can trap the electrons, an example of a double-dot device is shown in Figure 2. On the left is the cross-sectional view of this device and the electrostatic potential designed to trap individual electrons. This device has two plunger gates, and electrons are trapped under each plunger gate. Since an electron carries a negative charge, increasing P_1 or P_2 voltage deepens the potential level at the dot 1 or 2 (from the black solid line to the black dashed line), respectively. Then, we can increase/decrease the number of electrons in the dot by adjusting its corresponding plunger gate voltage to raise/lower the energy potential. However, adjusting the plunger gate voltage also has an unwanted effect, the cross-capacitance, which will affect the potential at the nearby dots by a non-negligible amount. For example, the increased P_2 voltage also pushes the potential level of dot 1 further to the red dashed line, which is deeper than the desired black dashed level.

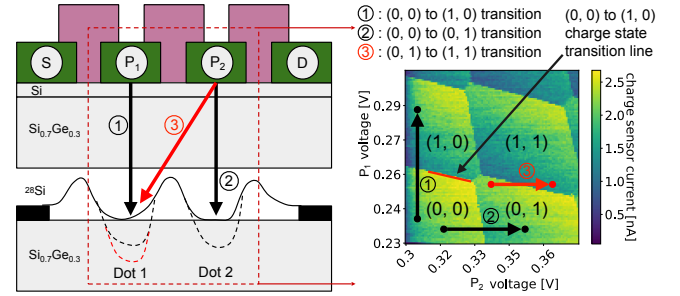


Figure 2: Example charge stability diagram of a double quantum dot device

Charge Stability Diagram The effect of nearby plunger gates on the charge state can be visualized in a Charge Stability Diagram (CSD). The CSD of the example double-dot device is on the right of Figure 2. A CSD represents the charge sensor current as a function of two adjacent plunger gate voltages. Charge state transition lines can be observed in a CSD, separating the diagram into four main regions, each representing a distinct charge state. The charge state of a dot is simply the number of electrons trapped in a dot. In the case of a double dot, we use a tuple (N_1, N_2) to represent the double dot’s charge state collectively, where N_1 is the number of electrons in dot 1, and N_2 is the number of electrons in dot 2. The charge states of the four regions are labeled in Figure 2. Assuming we start from the $(0, 0)$ region, as P_2 voltage is increased, the charge state changes from $(0, 0)$ to $(0, 1)$, corresponding to the arrow ② in Figure 2 (right), meaning that the increased P_2 voltage traps an electron in dot 2. Similarly, increasing the P_1 voltage achieves transition from $(0, 0)$ to $(1, 0)$, corresponding to the arrow ① in Figure 2, traps an electron in dot 1.

The cross-capacitance effect is indicated by the slopes of the charge state transition lines in the CSD. Assuming we start from a point in the $(0, 1)$ region and close to the $(1, 1)$ region, then due to the sloped $(0, 1)$ to $(1, 1)$ transition line, an increment in P_2 voltage may transit the state to $(1, 1)$, represented by the short arrow ③ in Figure 2. This is an undesirable effect, and ideally, plunger gate voltages should independently control the charge state of its corresponding dot.

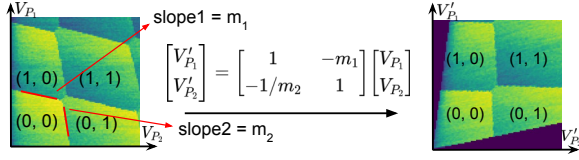


Figure 3: Virtual gate example

2.3 Virtual Gate

To resolve the aforementioned cross-capacitance nonideality and realize precise orthogonal control over individual quantum dots, a virtual gate technique is introduced to compensate for the non-negligible side effects from nearby plunger gates. A dot's virtual gate voltage is a linear combination of nearby physical gate voltages with carefully determined coefficients such that the linear combination of them realizes the ideal "one-to-one" control of the dot's potential, where one can adjust the energy level of the target dot without affecting adjacent dots.

Figure 3 shows an example of virtual gates for a double quantum dot device. On the left is the original CSD. V_{P1} and V_{P2} are the actual voltages on plunger gates 1 and 2. The virtual gate voltages V'_{P1} and V'_{P2} are the linear combination of the actual gate voltages:

$$\begin{bmatrix} V'_{P1} \\ V'_{P2} \end{bmatrix} = \begin{bmatrix} 1 & \alpha_{12} \\ \alpha_{21} & 1 \end{bmatrix} \begin{bmatrix} V_{P1} \\ V_{P2} \end{bmatrix}$$

where the 2x2 matrix is the virtualization matrix. The process of establishing virtual gates is equivalent to finding α_{12} and α_{21} in the virtualization matrix.

The values of α_{12} and α_{21} can be computed based on the slopes of the transition lines - the boundaries between different charge state areas in a CSD. Suppose m_1 and m_2 are the slopes of $(0, 0) \rightarrow (0, 1)$ and $(0, 0) \rightarrow (1, 0)$ transition lines. Then we have

$$\alpha_{12} = -m_1, \quad \alpha_{21} = -1/m_2$$

The virtual gate extraction can be extended to a n -dot array by sequentially applying it to every pair of nearby plunger gates [13], and $n - 1$ sequentially executed extraction processes are needed for an n -dot array.

On the right of Figure 3 is the CSD with the virtual gates. The virtualization matrix defines an affine transformation that warps the space to orthogonalize the transition lines. The orthogonal transition lines in the virtualized space demonstrate "one-to-one" control—increasing one virtual plunger gate voltage only affects the charge state on its corresponding dot.

3 PREVIOUS WORK

Previous works for constructing the virtualization matrix are mostly obtaining the full CSD for each pair of nearby plunger gates and then applying computer vision techniques on the CSDs to extract the slopes of the transition lines [12, 13, 18, 32]. However, obtaining a full CSD is slow. Each data point (V_{P1}, V_{P2}) in the CSD requires physically setting the voltages of the two plunger gates to be V_{P1} and V_{P2} , waiting a dwell time, and then measuring the current in the charge sensor (shown as a function in Algorithm 1).

The major bottle neck is the dwell time in line 3. This delay is necessary to account for the heavy filtering on the wires applying the bias voltages to the device. While hardware improvements can reduce this dwell time, typical dwell times are often on the order of

Algorithm 1 Get charge sensor current

```

1: function GETCURRENT( $v_1, v_2$ )
2:   Set gate voltages to  $v_1, v_2$ 
3:   wait(dwellTime)
4:   return charge sensor current

```

milliseconds, meaning typically it takes minutes to record one CSD of two quantum dots. Due to this overhead, the CSD acquisition becomes significant overhead for tuning large-scale devices.

4 FAST VIRTUAL GATE EXTRACTION

This paper aims to accelerate the virtual gate extraction so that the electron spin qubits can be quickly established for further experiments. Our optimization is based on a key observation that there is significant redundancy in the existing computer vision-based virtual gate extraction process. We aim to locate the transition lines using as few data points in the CSD as possible. Then, in the actual experiments, we only need to probe very few data points to reconstruct the virtualization matrix.

4.1 Redundancy in Full CSD Data

Since constructing the virtualization matrix depends on the slopes of the transition lines, only the data points near the transition lines will contribute to the calculation. Ideally, to find the slope of a line, if there were no noise in the experiment, two data points along the line would be enough to determine the slope of one transition line. Although there is inevitable noise in real experiments and more data points are required to precisely determine the transition lines, a lot of data points in the CSD are not necessary since the transition lines and their nearby data points are only a small portion of the entire CSD.

4.2 Locating Critical Region

To eliminate the redundancy, the first step of our method is to locate the region where the transition lines will lie in roughly. We first observe that the transition lines have negative slopes, and the slope of the $(0, 0) \rightarrow (0, 1)$ transition line is steeper than the $(0, 0) \rightarrow (1, 0)$ transition line. This observation can be validated by the physics capacitance model for quantum dots [6]. This prior knowledge suggests that if we know two endpoints (one on the $(0, 0) \rightarrow (1, 0)$ transition line and one on the $(0, 0) \rightarrow (0, 1)$ transition line), both transition lines are within the right triangular area defined by these two points. An example is shown in Figure 4. If the two red points are known, the highlighted right triangular region can cover both transition lines, and we only need to search in this region to locate the rest of the points on the lines. We name the points defining

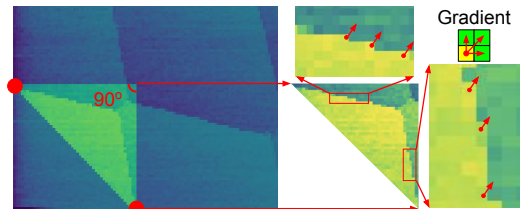


Figure 4: The highlighted area confines both transition lines. Points on the transition lines have a larger gradient in the positively tilted direction

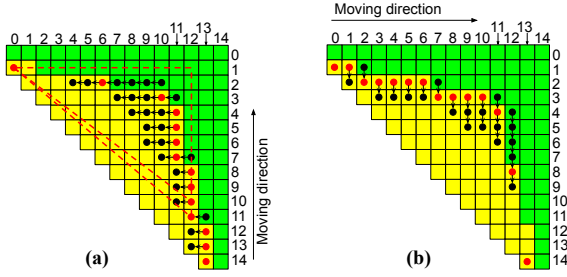


Figure 5: (a) Row-major sweep, (b) Column-major sweep

the triangle “anchor points”. We assume that the anchor points are known for now and will introduce how to find them later in Section 4.4.

4.3 Searching in the Critical Region

4.3.1 Gradient-Based Feature. After locating the critical region, we use a gradient-based feature to locate the points on the transition lines inside the triangular area. Note that a transition line indicates a sharp change in charge sensor current across the line. This feature results in a positively tilted gradient at the transition points, as shown on the top right of Figure 4. We use this positively tilted gradient as the feature to find the transition lines. For a point in the voltage space, we represent its positively tilted gradient by the sum of its differences in charge sensor current with the right and upper-right pixels, which we define as the feature gradient. Note the pixel here refers to the voltage granularity, not the actual image pixel. On real devices, assuming the gate voltages are at (V_{P_1}, V_{P_2}) , to get the charge sensor current at the right and upper-right pixels, the gate voltages are sequentially adjusted to $(V_{P_1}, V_{P_2} + \delta)$ and $(V_{P_1} + \delta, V_{P_2} + \delta)$, where δ is the voltage granularity that defines the pixel size. The procedure for computing gradient at gate voltages (v_1, v_2) is shown in Algorithm 2.

4.3.2 Search for Transition Lines. Using the gradient-based feature above, our method will locate points on the charge state transition lines in the triangular region by performing a bottom-to-top row-major sweep and a left-to-right column-major sweep. During the sweeps, the triangular region is dynamically shrunk to ensure only points near the transition lines are probed.

The row-major sweep goes through the points in a row-major fashion. As shown in Figure 5 (a), the sweep iterates through the rows from bottom to top. The upper left anchor point is fixed, and the lower right anchor point moves up. At each row, it probes the segment of points within the triangular region. We use a point’s pixel center to determine whether it’s inside the triangular region. GETGRADIENT is applied to each point in the segment to calculate their feature gradient, and the point with the maximum feature gradient is saved as a charge state transition point. We then update the lower anchor point to be the newly found charge state transition point—effectively shrinking the triangular area. Figure 5 (a)

Algorithm 2 Compute feature gradient

```

1: function GETGRADIENT( $v_1, v_2$ )
2:    $c \leftarrow \text{GETCURRENT}(v_1, v_2)$ 
3:    $c_{\text{Right}} \leftarrow \text{GETCURRENT}(v_1, v_2 + \delta)$ 
4:    $c_{\text{UpperRight}} \leftarrow \text{GETCURRENT}(v_1 + \delta, v_2 + \delta)$ 
5:   return  $(c - c_{\text{Right}}) + (c - c_{\text{UpperRight}})$ 

```

demonstrates the row-major sweep. The grid represents the voltage space. The points probed are marked with spots. The red spots represent the anchor points on each row. The black spots represent the points probed but not saved. The small black arrows on each row represent the order and direction of the sweep. As an example, when sweeping row 10, the following procedures are applied:

1. The lower anchor point is currently at (11, 12) (row, column), which is the charge state transition point located at row 11. The other fixed anchor point is at (1, 0). These two anchor points define the right triangular region whose vertices are (1, 0), (1, 12), (11, 12). The edges of the triangle are the dashed red lines in Figure 5 (a).
2. The sweep goes through points (10, 12) and (10, 11), the only two points within the triangular area on row 10, and then computes their feature gradients. Here, point (10, 12) has a larger feature gradient, so it is saved as a charge state transition point.
3. Update the lower anchor point from (11, 12) to the newly found charge state transition point (10, 12).

The row-major sweep performs these procedures on each row until it reaches the row of the fixed anchor point (row 1 in 5a). Notice that the shrinking triangular region keeps the search near the transition lines, probing only a few points at each row where charge state transition is most likely to occur.

The row-major sweep is effective in locating points on the $(0, 0) \rightarrow (0, 1)$ transition line but less effective in locating points on the $(0, 0) \rightarrow (1, 0)$ transition line because it is less orthogonal to the sweeping direction. As shown in Figure 5 (a), when the row-major sweep reaches the $(0, 0) \rightarrow (1, 0)$ transition region (row 3 and above), the segment inside the triangular region becomes relatively long (e.g., the segment at row 2 in Figure 5 (a) is longer than the segments at row 5-13), making the search more susceptible to noise. It is also possible in this region that a falsely located point deviates the triangular region from the transition line, producing a series of falsely located points. To locate equally accurate points on the $(0, 0) \rightarrow (1, 0)$ transition line, a column-major sweep is performed in analogs to the row-major sweep. The column-major sweep is performed from left to right and is demonstrated in Figure 5 (b). It is similar to the previous row-major sweep, with only the row and column exchanged.

A post-processing step shown in Figure 6 is included to filter erroneous points. The upper left image in Figure 6 shows all points located by the two sweeps on an example voltage space, where

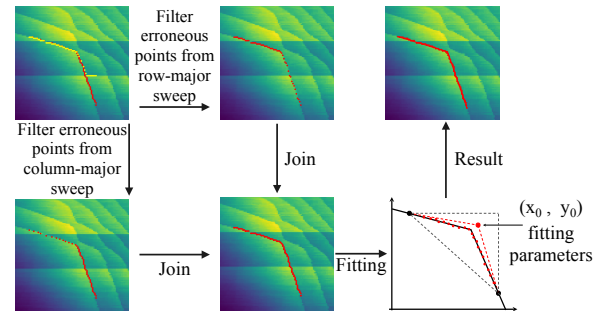


Figure 6: Post processing procedure running on example data

Algorithm 3 Sweeps + Post-processing

```

1: procedure POSTPROCESS(points)
2:   filteredPoints1 = { (x, y) |  $\forall (x', y') \in \text{points}, y \leq y'$  }
3:   filteredPoints2 = { (x, y) |  $\forall (x', y) \in \text{points}, x \leq x'$  }
4:   return filteredPoints1  $\cup$  filteredPoints2
5: procedure SWEEPS((x1, y1), (x2, y2))
6:   anchorPoint1, anchorPoint2  $\leftarrow$  (x1, y1), (x2, y2)
7:   transitionPoints = []
8:   for i  $\leftarrow$  y2 + 1 to y1 - 1 do
9:     P  $\leftarrow$  points inside the triangular region at row i
10:    (x*, y*)  $\leftarrow$   $\underset{(x,y) \in P}{\text{argmax}}$  GETGRADIENT(y, x)
11:    append (x*, y*) to transitionPoints
12:    anchorPoint2  $\leftarrow$  (x*, y*)
13:   anchorPoint2  $\leftarrow$  (x2, y2)
14:   for j  $\leftarrow$  x1 + 1 to x2 - 1 do
15:     P  $\leftarrow$  points inside the triangular region at column j
16:     (x*, y*)  $\leftarrow$   $\underset{(x,y) \in P}{\text{argmax}}$  GETGRADIENT(y, x)
17:     append (x*, y*) to transitionPoints
18:     anchorPoint1  $\leftarrow$  (x*, y*)
19:   return POSTPROCESS(transitionPoints)

```

the red points are produced by the row-major sweep and the yellow points are produced by the column-major sweep. Erroneous points are likely to be produced when the row-major sweep attempts to locate points in the (0, 0)→(1, 0) transition region and when the column-major sweep attempts to locate the points in the (0, 0)→(0, 1) transition region because they will sweep more points in a row/column and are more error-prone. The post-processing step produces two filtered sets of points and joins them together. For the first set, we only keep the point with the smallest *y* value at each *x* position (i.e., the lowest point in each column). This is designed to exclude erroneous red points using the yellow points below them. Similarly, for the second set, we only keep the point with the smallest *x* value at each *y* position (i.e., the leftmost point in each row). By joining the two filtered sets, we get a comprehensive set of points on both transition lines with minimal erroneous points. The procedures of row-major, column-major sweeps, and post-processing are summarized in Algorithm 3.

4.3.3 Slope Extraction. After the transition line points are located, we parameterize a 2-piece-wise linear shape by the two initial anchor points and the intersecting point of the two transition lines, where the coordinates of the intersecting point are the fitting parameters. We use SciPy's curve fitting function to find the optimal position of the intersecting point [25]. The slopes of the transition lines are then computed using the intersecting point and the initial anchor points.

4.4 Preprocessing for Anchor Points

Finally, we introduce our preprocessing method to locate the two initial anchor points. We first probe ten equally spaced points spanning the diagonal from the lower left to the upper right. Then we apply the following masks along *x* and *y* directions, starting from the brightest point found in the previous step or 10% width and height, whichever is more distant from the lowest and leftmost point. These masks are designed to compute a positively sloped gradient across three pixels, a more noise-resilient indicator of charge

state transition than the feature gradient.

$$Mask_x = \begin{bmatrix} 1 & 1 & -3 & -4 & -4 \\ 2 & 2 & 0 & -2 & -2 \\ 4 & 4 & 3 & -1 & -1 \end{bmatrix}, \quad Mask_y = \begin{bmatrix} -1 & -2 & -4 \\ -1 & -2 & -4 \\ 3 & 0 & -3 \\ 4 & 2 & 1 \\ 4 & 2 & 1 \end{bmatrix}$$

We sweep the masks along their corresponding axis and sum the entries of their element-wise product with the pixel values. The resulting arrays are then multiplied element-wise by the 1D Gaussian distribution. The point with the maximum value in each array is used as an initial anchor point.

5 EVALUATION

In this section, we evaluate the proposed fast virtual gate extraction method with respect to the speed and accuracy of experimental data of real quantum dot devices.

5.1 Experiment Setup

Benchmark We adopt the 12 experimentally measured charge stability diagrams (CSDs) in the qflow dataset version 2 [35]. Note that this has covered all the real experimental data from this dataset. These CSDs were measured in the double-dot configuration on a triple-dot Si/SiGe device fabricated on a 300mm industrial line [20, 35]. The diagrams are cropped to focus on the 50% width and height region where the (0, 0), (0, 1), (1, 0), (1, 1) charge state regions are located. The final data have pixel resolutions ranging from 63 × 63 (126×126 before cropping) to 200×200 (401×401 before cropping).

Metrics We use the rate of successfully finding the transition lines to indicate the applicability of the proposed algorithm and the baseline. We use the total runtime to evaluate the overall speedup. We also collect the number of data points probed because this is the bottleneck in the entire virtual gate extraction process, and our speed is achieved by reducing the number of probed data points.

Implementation We prototype the proposed virtual gate extraction algorithm in Python and simulate the experiment setup using the data from real experiments. When the proposed algorithm needs to obtain a data point with a specific voltage combination, it will call a simulated GETCURRENT function with the voltage coordinates. The GETCURRENT function will return a current from a CSD in the dataset to simulate an experiment on that device. The GETCURRENT function uses a dwelling time of 50ms, which is a typical time delay for charge-sensor-based devices [30]. Since the benchmark dataset does not provide reference virtual gates, we plot the final affine transformed diagram for virtualization and manually examine whether the virtual gate extraction is successful or not.

Baseline Our baseline is the Hough transform with Canny edge detection, an existing technique used for virtual gate automation [12, 18] implemented using the opencv-python [3]. The baseline first obtains a full CSD by calling the simulated GETCURRENT on all voltage combinations before image processing.

5.2 Result

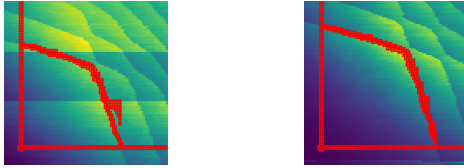
Table 1 shows the results of applying the proposed method and the baseline on all the benchmarks. Overall, our fast virtual gate extraction method can successfully find the transition lines and then construct the virtual gate in 10 out of the 12 benchmarks, while

Table 1: Result Summary

Benchmark		Success/Fail		Number/percentage of points probed		Total runtime		Speedup
CSD Index	Size	Fast Extraction	Baseline	Fast Extraction	Baseline	Fast Extraction	Baseline	Speedup
1	200×200	Fail	Fail	1753 (4.38%)	40000 (100%)	87.94s	2005.12s	N/A
2	200×200	Fail	Fail	1692 (4.23%)	40000 (100%)	84.89s	2004.98s	N/A
3	63×63	Success	Success	643 (16.2%)	3969 (100%)	32.26s	198.96s	6.16×
4	63×63	Success	Success	679 (17.1%)	3969 (100%)	34.06s	198.97s	5.84×
5	63×63	Success	Success	484 (12.19%)	3969 (100%)	24.28s	198.94s	8.19×
6	100×100	Success	Success	1002 (10.02%)	10000 (100%)	50.27s	501.25s	9.97×
7	100×100	Success	Fail	985 (9.85%)	10000 (100%)	49.42s	501.26s	10.14×
8	100×100	Success	Success	1179 (11.79%)	10000 (100%)	59.14s	501.26s	8.48×
9	100×100	Success	Success	974 (9.74%)	10000 (100%)	48.86s	501.27s	10.26×
10	100×100	Success	Success	1054 (10.54%)	10000 (100%)	52.88s	501.27s	9.48×
11	100×100	Success	Success	927 (9.27%)	10000 (100%)	46.5s	501.26s	10.78×
12	200×200	Success	Success	2067 (5.17%)	40000 (100%)	103.69s	2005.02s	19.34×

the baseline Hough transform succeeds on only 9 benchmarks. We manually checked the two benchmarks we failed on. The reason is that those two devices have too much noise in the CSDs and both our method and the baseline fail to locate the transition lines. We also investigated the case CSD 7 where the baseline fails while our method succeeds. We found that the edge detection in the baseline could not locate enough points to establish the line. Meanwhile, the column-major sweep in our method successfully located the line.

In terms of the virtual gate extraction speed, our fast extraction method can achieve a speedup ranging from 5.84× to 19.34× against the baseline. The speed-up mostly comes from the reduction in the number of data points probed as our method adaptively locates the critical region and only probes a small number of actual gate voltage configurations. The baseline probes every point in the space to fill a full CSD, while our method only probes 10% on average as shown in the middle of Table 1. As an example, Figure 7 shows the data points probed in CSD 6 and CSD 10 by our method, and the probed data points for other benchmarks are similar. Note that the points are mostly scattered around the two transition lines, with additional points probed to determine the initial anchor points. In summary, our fast virtual gate extraction method outperforms the Hough transform baseline with faster virtual gate extraction, fewer experimental data requirements, and even higher accuracy.

**Figure 7: Data points probed in benchmark CSD 6 and 10**

ACKNOWLEDGMENTS

This work was supported in part by NSF CAREER Award 2338773.

REFERENCES

- [1] Tim Baart et al. 2016. Single-spin CCD. *Nature Nanotechnology* 11, 4 (2016), 330–334.
- [2] Francesco Borsio et al. 2024. Shared control of a 16 semiconductor quantum dot crossbar array. *Nature Nanotechnology* 19, 1 (2024), 21–27.
- [3] G. Bradski. 2000. The OpenCV Library. (2000).
- [4] Guido Burkard et al. 2023. Semiconductor Spin Qubits. *Reviews of Modern Physics* 95, 2 (2023), 025003.
- [5] David DiVincenzo. 2000. The physical implementation of quantum computation. *Fortschritte der Physik: Progress of Physics* 48, 9-11 (2000), 771–783.
- [6] Ronald Hanson et al. 2007. Spins in few-electron quantum dots. *Reviews of modern physics* 79, 4 (2007), 1217–1265.
- [7] Patrick Harvey-Collard et al. 2018. High-Fidelity Single-Shot Readout for a Spin Qubit via an Enhanced Latching Mechanism. *Physical Review X* 8, 2 (2018), 021046.
- [8] Jonathan Huang et al. 2023. High-fidelity operation and algorithmic initialisation of spin qubits above one kelvin. (2023). <https://arxiv.org/abs/2308.02111>
- [9] Wister Huang et al. 2019. Fidelity benchmarks for two-qubit gates in silicon. *Nature* 569, 7757 (2019), 532–536.
- [10] Intel. 2023. *Intel's New Chip to Advance Silicon Spin Qubit Research for Quantum Computing*.
- [11] Daniel Loss and David DiVincenzo. 1998. Quantum computation with quantum dots. *Physical Review A* 57, 1 (1998), 120–126.
- [12] Adam Mills et al. 2019. Computer-automated tuning procedures for semiconductor quantum dot arrays. *Applied Physics Letters* 115, 11 (2019), 113501.
- [13] Adam Mills et al. 2019. Shuttling a single charge across a one-dimensional array of silicon quantum dots. *Nature Communications* 10, 1 (2019), 1063.
- [14] Adam Mills et al. 2022. High-Fidelity State Preparation, Quantum Control, and Readout of an Isotopically Enriched Silicon Spin Qubit. *Physical Review Applied* 18, 6 (2022), 064028.
- [15] Adam Mills et al. 2022. Two-qubit silicon quantum processor with operation fidelity exceeding 99%. *Science Advances* 8, 14 (2022), eabn5130.
- [16] Hyungil Moon et al. 2020. Machine learning enables completely automatic tuning of a quantum device faster than human experts. *Nature Communications* 11, 1 (2020), 4161.
- [17] Akito Noiri et al. 2022. Fast universal quantum gate above the fault-tolerance threshold in silicon. *Nature* 601, 7893 (2022), 338–342.
- [18] Giovanni Oakes et al. 2020. Automatic virtual voltage extraction of a 2x2 array of quantum dots with machine learning. *arXiv preprint arXiv:2012.03685* (2020).
- [19] Stephan Philips et al. 2022. Universal control of a six-qubit quantum processor in silicon. *Nature* 609, 7929 (2022), 919–924.
- [20] Ravi Pillarisetty et al. 2021. Si MOS and Si/SiGe quantum well spin qubit platforms for scalable quantum computing. In *2021 IEEE International Electron Devices Meeting (IEDM)*. 14.1.1–14.1.4.
- [21] Kenta Takeda et al. 2024. Rapid single-shot parity spin readout in a silicon double quantum dot with fidelity exceeding 99%. *npj Quantum Information* 10, 1 (2024), 22.
- [22] Alexei Tyryshkin et al. 2006. Coherence of spin qubits in silicon. *Journal of Physics: Condensed Matter* 18, 21 (2006), S783–S794.
- [23] Alexei Tyryshkin et al. 2012. Electron spin coherence exceeding seconds in high-purity silicon. *Nature Materials* 11, 2 (2012), 143–147.
- [24] Cornelis Van Diepen et al. 2018. Automated tuning of inter-dot tunnel coupling in double quantum dots. *Applied Physics Letters* 113, 3 (2018), 033101.
- [25] Pauli Virtanen et al. 2020. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature methods* 17, 3 (2020), 261–272.
- [26] Christian Volk et al. 2019. Loading a quantum-dot based “Qubyte” register. *npj Quantum Information* 5, 1 (2019), 29.
- [27] Thomas Watson et al. 2018. A programmable two-qubit quantum processor in silicon. *nature* 555, 7698 (2018), 633–637.
- [28] Xiao Xue et al. 2022. Quantum logic with spin qubits crossing the surface code threshold. *Nature* 601, 7893 (2022), 343–347.
- [29] Jun Yoneda et al. 2018. A quantum-dot spin qubit with coherence limited by charge noise and fidelity higher than 99.9%. *Nature nanotechnology* 13, 2 (2018), 102–106.
- [30] David Zajac. 2018. *Single Electron Spin Qubits in Silicon Quantum Dots*. Ph.D. Dissertation.
- [31] David Zajac et al. 2018. Resonantly driven CNOT gate for electron spins. *Science* 359, 6374 (2018), 439–442.
- [32] Joshua Ziegler et al. 2023. Automated extraction of capacitive coupling for quantum dot systems. *Physical Review Applied* 19, 5 (2023), 054077.
- [33] Joshua Ziegler et al. 2023. Tuning arrays with rays: Physics-informed tuning of quantum dot charge states. *Physical Review Applied* 20, 3 (2023), 034067.
- [34] Anne Zwerger et al. 2022. Qubits made by advanced semiconductor manufacturing. *Nature Electronics* 5, 3 (2022), 184–190.
- [35] Justyna Zwolak et al. 2018. QFlow lite dataset: A machine-learning approach to the charge states in quantum dot experiments. *PLoS One* 13, 10 (2018), e0205844.

G-kway: Multilevel GPU-Accelerated k -way Graph Partitioner

Wan Luan Lee
University of Wisconsin-Madison
Madison, USA
wanluan.lee@wisc.edu

Dian-Lun Lin
University of Wisconsin-Madison
Madison, USA
dianlun.lin@wisc.edu

Tsung-Wei Huang
University of Wisconsin-Madison
Madison, USA
tsung-wei.huang@wisc.edu

Shui Jiang
The Chinese University of Hong Kong
Hong Kong, China
sjiang22@cse.cuhk.edu.hk

Tsung-Yi Ho
The Chinese University of Hong Kong
Hong Kong, China
tyho@cse.cuhk.edu.hk

Yibo Lin
Peking University
Beijing, China
yibolin@pku.edu.cn

Bei Yu
The Chinese University of Hong Kong
Hong Kong, China
byu@cse.cuhk.edu.hk

Abstract

Graph partitioning is important for the design of many CAD algorithms. However, as the graph size continues to grow, graph partitioning becomes increasingly time-consuming. To overcome these challenges, we propose G-kway, an efficient multilevel GPU-accelerated k -way graph partitioner. G-kway introduces an effective union find-based coarsening and a novel independent set-based refinement algorithm to significantly accelerate both the coarsening and uncoarsening stages. Experimental results have shown that G-kway outperforms both the state-of-the-art CPU-based and GPU-based parallel partitioners with an average speedup of 8.6 $\times$ and 3.8 $\times$, respectively, while achieving comparable partitioning quality.

ACM Reference Format:

Wan Luan Lee, Dian-Lun Lin, Tsung-Wei Huang, Shui Jiang, Tsung-Yi Ho, Yibo Lin, and Bei Yu. 2024. G-kway: Multilevel GPU-Accelerated k -way Graph Partitioner. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3649329.3656238>

1 Introduction

Graph partitioning is important for the design of efficient computer-aided design (CAD) algorithms because it allows an algorithm to break down a problem into smaller and manageable pieces. Among various partitioning frameworks, *multilevel partitioning* is the most popular for large-scale graphs due to its high partitioning quality and fast runtime. A typical multilevel partitioner iteratively coarsens the original graph into a smaller representation. When the graph becomes small enough, the partitioner iteratively restores the graph back to a larger one, followed by a refinement algorithm.

However, as the size of circuit graphs continues to increase, graph partitioning becomes time-consuming. To alleviate the long runtime, existing partitioners [7] have leveraged multi-core CPUs to parallelize the partitioning algorithm. Despite some runtime improvements, the speedup is typically limited to only 8–16 CPU threads [7]. On the other hand, modern GPUs offer a massive amount of parallelism and memory bandwidth that present an opportunity to accelerate graph partitioning to a new performance degree. For instance, [1] proposes a CPU-GPU-hybrid multilevel graph partitioner that dynamically performs the work on either the GPU or CPU. However, their approach requires frequent data transfers between CPU and GPU, resulting in significant runtime overhead. To address this problem, GKSG [2] performs the entire graph partitioning on GPU. However, their performance is far from optimal due to limited parallelism. Specifically, GKSG’s refinement algorithm can only move a few vertices (e.g., 8) in parallel due to limited GPU memory, as it counts on an exponential enumeration to find a valid refinement. Furthermore, GKSG’s coarsening algorithm requires many sequential matching iterations, largely underutilizing the massive parallelism in GPU. As a consequence, GKSG reported only an average 1.9 $\times$ speedup over a CPU-parallel partitioner [2].

To overcome these problems, we propose G-kway, a new GPU-accelerated k -way graph partitioner. G-kway introduces a union find-based coarsening algorithm that can merge many vertices simultaneously to substantially reduce the number of coarsening levels while keeping good partitioning quality. Additionally, G-kway introduces a new independent set-based refinement algorithm that can refine many vertices in parallel, largely reducing the number of refinement iterations. To further optimize the performance of our GPU kernels, G-kway applies various modern GPU optimization techniques, such as pinned memory access and warp-level primitives.

We have evaluated the performance of G-kway on industrial circuit graphs and compared our results with two state-of-the-art parallel graph partitioners, CPU-based mt-metis [7] and GPU-based GKSG [2]. On average, experimental results have shown that G-kway outperforms 32-threaded mt-metis and GKSG by 8.6 $\times$ and 3.8 $\times$ faster, respectively, with comparable cut sizes.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

DAC '24, June 23–27, 2024, San Francisco, CA, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0601-1/24/06...\$15.00

<https://doi.org/10.1145/3649329.3656238>

2 Problem Definition and Notation

Given an undirected graph, $G = (V, E)$, where V is a set of vertices, and E is a set of edges. Each element in E is of the form $e = (u, v)$ which represents the connection between u and v in V . For a vertex $v \in V$, we denote the weight of v by W_v , while for an edge $e \in E$, we denote the weight of e by W_e . For a vertex $v \in V$, its adjacent vertex set is denoted as $adj(v)$. Given k , if $P = \{p_1, p_2, \dots, p_k\}$ is a disjoint partition of V , we call P a k -way partition. For $v \in V$, we define $P(v) = i$ if $v \in p_i$. We define the cut size as $\sum_{e=(u,v) \in E, P(u) \neq P(v)} W_e$. Cut size is widely used for evaluating the quality of a partition since it represents the interconnect complexity among partitions. The partition weight of p_i is defined as $W_{p_i} = \sum_{v \in p_i} W_v$. The goal of the graph partition problem is to find a k -way partition that satisfies the balance constraint while minimizing the cut size. The balance constraint limits the maximum weight of p_i as $W_{p_i} \leq (1+\epsilon) \frac{\sum_{v \in V} W_v}{k}$, where $0 < \epsilon \ll 1$ and ϵ is the imbalance ratio given by applications.

3 GPU Multilevel k -way Partitioner

Figure 1 shows the overview of G-kway that consists of three main stages: *coarsening*, *initial partition*, and *uncoarsening*.

- **Coarsening.** The goal is to coarsen the graph into a smaller representation level by level while preserving the original graph's structure. The coarsening level continues until the graph size becomes smaller than a certain threshold (typically $\frac{|V|}{20 \times (\log_2(k))}$). We develop a *union find-based coarsening* that substantially reduces the number of coarsening levels while still maintaining a good representation of the original graph structure.
- **Initial partition.** The goal is to create an initial partition from the coarsest graph. We utilize single-threaded Metis [5] for the initial partition. Since the coarsest graph is much smaller than the original graph, the initial partition stage is very fast and does not benefit much from CPU/GPU parallelism.
- **Uncoarsening.** The goal is to iteratively restore the coarsened graph back to its previous graph and reduce the cut size of a coarsened graph by moving each vertex to a partition (i.e., refinement). The uncoarsening level continues until the graph size is the same as the original graph. We develop an efficient *independent set-based refinement* algorithm that reduces the cut size by moving many vertices among partitions in parallel.

Multilevel graph partitioning requires many iterative control-flow operations performed on the CPU to determine termination. Such frequent CPU-GPU data transfers can result in significant runtime overhead. To address this issue, G-kway utilizes CUDA pinned memory for control-flow data to avoid swapping out memory to disk by the operating system. In both coarsening and uncoarsening stages, we utilize modern warp-level primitives for our GPU kernels to further optimize the performance. In terms of graph storage, G-kway utilizes the commonly used compressed sparse row (CSR) data structure [2] for efficient GPU computing.

3.1 Union Find-based Coarsening with Scoring

Most existing parallel multilevel graph partitioners such as GKSG [2] implement a parallel Heavy Edge Matching (HEM) algorithm that finds matching pairs to coarsen the original graph. Specifically, each vertex searches for a neighbor with the heaviest edge to form a matching pair and coarsen the two vertices into a coarsened vertex. However, this matching algorithm requires both

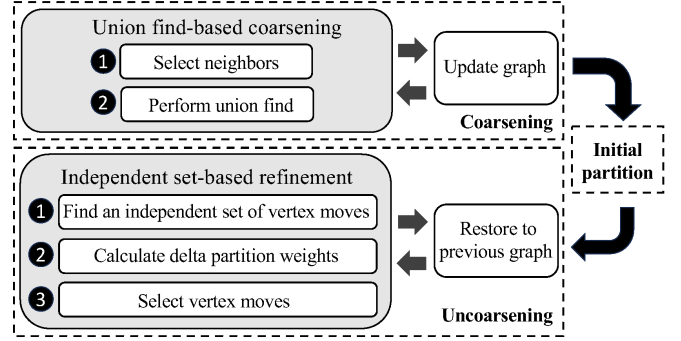


Figure 1: Overview of G-kway that consists of three main stages: coarsening, initial partition, and uncoarsening.

vertices to choose each other. If both vertices have many neighbors connected with the same heaviest edges, they may choose different neighbors for matching, preventing the formation of matching pairs and leaving many vertices unable to match. The unmatched vertices continue to search with their remaining neighbors in the next matching iteration. Such an iterative algorithm largely underutilizes the massive parallelism in GPU. Furthermore, GKSG can only coarsen two vertices per matching pair, thus requiring many coarsening levels until the size of the coarsened graph is smaller than the threshold. Figure 2 shows the comparison between GKSG's coarsening algorithm and ours. As shown in (a), GKSG can only match v_1 and v_2 in the first iteration, leaving the unmatched vertices v_3 and v_4 for the next matching iteration.

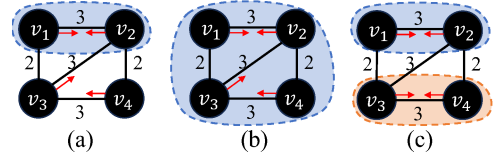


Figure 2: Examples of three coarsening methods for one iteration, including (a) Heavy Edge Matching (HEM) by GKSG, (b) Union find-based coarsening without scoring, and (c) Union find-based coarsening with scoring. Each vertex has a red arrow pointing to its selected neighbor. Vertices circled in the same color are coarsened into a coarsened vertex.

To address these issues, our initial solution is to merge vertices into subsets and coarsen all vertices in the same subset into a coarsened vertex. Each vertex finds a neighbor with the heaviest edge. If that neighbor belongs to another subset, we merge the vertex into the same subset. This union find-based strategy eliminates the need for iteratively searching neighbors to match, ensuring each vertex can find a neighbor to merge in only one iteration. Also, since we merge multiple vertices per subset, it requires much fewer coarsening levels than GKSG. However, this strategy can cause highly imbalanced subsets that largely impact refinement quality in the next stage since many vertices may all be merged into the same subset. As shown in 2 (b), v_1 and v_3 choose v_2 , v_2 chooses v_1 , and v_4 chooses v_3 . While the solution allows each vertex to find a neighbor in one iteration, all vertices are eventually merged together.

To this end, we propose a union find-based coarsening with scoring. Each vertex calculates the score for each connected edge and

selects a neighbor with the highest score to form a subset. Specifically, when a vertex u has multiple neighbors with the same heaviest edge, we prioritize the neighbor of u with the lower degree by assigning a higher score to the edge connected to this neighbor. Figure 2 (c) shows our union find-based coarsening with scoring. v_3 selects v_4 instead of v_2 since v_4 has lower degree than v_2 , resulting in two balanced subsets. Our coarsening algorithm consists of two steps: *select neighbors* and *perform union find*.

3.1.1 Select neighbors. We first find a neighbor connected by the edge with the highest score for each vertex. Given a source vertex u , we define the score of its edge (u, v) as $s(u, v) = c \times W_{(u, v)} - \text{degree}(v)$, where $\text{degree}(v)$ is the number of neighbors of v , c is a constant no less than the maximum degree of the graph, and $W_{(u, v)}$ is the edge weight of $e = (u, v)$. Algorithm 1 shows our neighbor selection algorithm which leverages an efficient *Warp segmentation* technique [6]. We assign 32 consecutive vertices and their edges to each GPU warp. Each GPU thread then processes an edge (u, v) by finding the source vertex (line 5) and calculating the edge score (line 6). Next, threads whose assigned edges belong to the same source vertex perform parallel reduction to identify the edge with the highest score (line 8). During reduction, we employ CUDA warp-level primitives, `__shfl_up_sync`, to efficiently exchange scores among threads in the same warp. Using the warp-level primitive allows threads in the same warp to share data through registers, which is much faster than through GPU global or share memory [6]. Finally, we map each thread to a vertex, and each thread is responsible for writing a vertex's neighbor connected by the highest-score edge to the array `selected_nbr` in the GPU's global memory (line 10).

3.1.2 Perform union find. After selecting the highest score neighbor for each vertex, we perform `union_find` to merge vertices into a subset. We maintain an additional array, `d_subset_ID`, to record each vertex's subset ID, where each vertex's subset ID is initialized to its vertex ID. We assign each vertex v_i to a GPU thread; then, each thread gets its assigned vertex's selected neighbor from the previous step stored in `selected_nbr` (line 15), and its vertex and selected neighbor's subset IDs from `d_subset_ID` (lines 16-17). Each thread then merges vertices by comparing its assigned vertex and the selected neighbor's subset ID and changing the larger ID to the smaller one (lines 18-21). At the end of each iteration, we employ CUDA warp-level voting primitives, `__any_sync`, to efficiently check if any thread in the warp updates the subset ID (line 23). We then repeat this process until no vertex's subset ID is updated. Finally, we coarsen vertices with the same subset ID into a coarsened vertex to derive the coarsened graph.

3.2 Independent Set-based Refinement

The goal of the refinement algorithm is to reduce the cut size by moving a vertex to a partition, seeking the maximum gain in cut size reduction. We define the gain of a vertex u for a partition p_i as $\text{gain}(u, p_i) = \text{ed}(u, p_i) - \text{id}(u)$, where $u \notin p_i$. $\text{id}(u)$ represents the internal degree of u , which is the sum of the weights of each edge (u, v) such that u and v are in the same partition. $\text{ed}(u, p_i)$ represents the external degree of u to partition p_i , which is the sum of the weights of each edge (u, v) such that v is in partition p_i . In refinement, we only consider moving a vertex at the partition boundary (i.e., one of its neighbors is located in a different partition).

Algorithm 1 Union find-based coarsening with scoring

```

1: /* select neighbors: assign 32 vertices and their edges to a GPU warp */
2: parallel for each thread in a warp {
3:   while (there are more edges to process) {
4:     get an edge  $e_i = (u, v)$  to process
5:     find the assigned edge's source vertex  $u$ 
6:      $s(u, v) \leftarrow c \times W_{(u, v)} - \text{degree}(v)$ 
7:     /* using __shfl_up_sync */
8:     reduce on the scores with threads have the same source vertex
9:   }
10:  write a vertex  $u$ 's selected neighbor to selected_nbr array
11: }
12: /* union find: assign each vertex  $v_i$  to a GPU thread  $T_i$  */
13: while (any threads is still updating) {
14:   parallel for each thread in a warp {
15:      $\text{nbr} \leftarrow \text{selected\_nbr}[v_i]$ 
16:      $v_i\_subset\_ID \leftarrow d\_subset\_ID[v_i]$ 
17:      $\text{nbr\_subset\_ID} \leftarrow d\_subset\_ID[\text{nbr}]$ 
18:     if ( $v_i\_subset\_ID > \text{nbr\_subset\_ID}$ ) then
19:       atomicMin(& $d\_subset\_ID[v_i]$ ,  $\text{nbr\_subset\_ID}$ )
20:     else if ( $v_i\_subset\_ID < \text{nbr\_subset\_ID}$ ) then
21:       atomicMin(& $d\_subset\_ID[\text{nbr}]$ ,  $v_i\_subset\_ID$ )
22:     /* using __any_sync */
23:     check if any thread in a warp updates subset_IDs
24:   }
25: }
```

Moving vertices not at the partition boundary cannot have positive gain, as $\text{ed}(u, p_i)$ is always zero.

To move multiple vertices in parallel while ensuring that the move results in the largest gain, GKSG's refinement algorithm enumerates all possible moves [2]. Each move represents a combination of vertices, where each vertex either moves to its destination partition or not. For example, to move eight vertices in parallel, GKSG will launch a GPU kernel with $2^8 \times 32$ threads to calculate 2^8 possible moves, where each move is verified by a GPU warp of 32 threads. This *exponential* enumeration algorithm limits the number of vertices that can be moved in parallel due to the limited GPU memory.

To overcome this problem, we propose an independent set-based refinement algorithm that can move many vertices in parallel. Our algorithm does not exponentially enumerate all possible moves, thus enabling much more parallelism without being constrained by GPU memory limitations. Algorithm 2 shows our refinement algorithm, which contains three steps: *find an independent set of vertex moves*, *calculate delta partition weights*, and *select vertex moves*. We iteratively perform our refinement algorithm until no vertex with positive gain can be moved.

3.2.1 Find an independent set of vertex moves. Moving multiple vertices in parallel is challenging. Even though each vertex has a positive gain, the overall cut size after all moves can remain or even increase due to interconnections among vertices. Furthermore, moving connected (i.e., adjacent) vertices in parallel requires expensive synchronization to keep updating gains. To address these issues, we find an independent set of vertices to move in parallel. We define each *vertex move* as $m_u^{\text{src}, \text{dst}}$, a struct that consists of a vertex ID (u), its source partition ID (src), its destination partition ID (dst), and the gain. We then use a move buffer to store vertex moves.

Algorithm 2 presents our independent set-based refinement algorithm. To find an independent set of vertex moves, we distribute each vertex in the graph to a GPU thread, where each GPU thread determines whether its vertex is at the partition boundary. If the vertex is at the boundary, the GPU thread finds a legal destination partition for that vertex (line 7).

We say a vertex has a legal destination partition if there exists one destination partition such that moving the vertex to that partition has a positive gain without violating the balance constraint. If a vertex has a legal destination partition, the GPU thread checks if any of its neighbors also have a legal destination partition (line 9). If no such neighbor exists, the GPU thread creates a vertex move for the vertex and inserts it into the move buffer (lines 10-12). Otherwise, we compare that vertex with its neighbors' IDs. We then only create a vertex move for the vertex with the smallest ID and insert it into the move buffer (lines 13-16). This organization ensures that no adjacent vertices are inserted into the move buffer.

Algorithm 2 Independent set-based refinement

```

1: while (true) {
2:   /* find an independent set of vertex moves */
3:   /* assign each vertex  $v_i$  to a GPU thread  $T_i$  */
4:   parallel for each thread {
5:     if  $v_i$  is not at a partition boundary then
6:       return
7:      $dst \leftarrow$  find a legal destination partition with the largest gain
8:     if ( $dst$  exists) then
9:        $nbors \leftarrow nbr$  in  $adj(v_i)$  has a legal destination partition
10:      if ( $nbors$  is empty) then
11:        create a vertex move for  $v_i$ 
12:        insert the vertex move to the move buffer
13:      else
14:        if ( $v_i.ID < \text{each } nbr.ID \text{ in } nbors$ ) then
15:          create a vertex move for  $v_i$ 
16:          insert the vertex move to the move buffer
17:    }
18:   if (the move buffer is empty) then
19:     return
20:   calculate delta partition weights /* Section 3.2.2 */
21:   select vertex moves /* Section 3.2.3 */
22: }
23: return

```

After finding an independent set of vertex moves, we need to select a subset of them such that applying those vertex moves still satisfies the balance constraint. However, finding the best subset still encounters the exponential enumeration problem (i.e., to select or not to select per vertex move). To address this challenge, we design a sequence-based strategy that first sorts each vertex move by gain to form a sequence and selects the longest sub-sequence of vertex moves that satisfies the balance constraint. While this strategy may not be the absolute best subset, selecting vertex moves from the largest gain ensures we prioritize the vertex moves that make a substantial contribution to overall cut size improvement. In the following sections, we present how to find that sub-sequence of vertex moves.

3.2.2 Calculate delta partition weights. In this step, we sort each vertex move by gain in descending order and calculate the delta partition weight of each vertex move to check the balance constraint.

We define the delta partition weight of a vertex move $m_u^{src,dst}$ for a partition p_i as follows:

$$\delta_i(m_u^{src,dst}) = \begin{cases} W_u, & i = dst \\ -W_u, & i = src \\ 0, & otherwise \end{cases}$$

We maintain a k -segment array, del_p_wgt , where each segment initially stores the delta partition weight of each vertex move for a partition. The segment size is the minimum of the total number of vertex moves and 1024. Since most modern GPUs have 1024 threads per GPU block, calculating more than 1024 vertex moves needs multiple blocks for each segment, which requires expensive synchronization across multiple blocks.

Figure 3 shows an example of our algorithm for six vertex moves with $k = 2$. Each element in del_p_wgt records the delta partition weight of each of the six vertex moves, where the first six elements (i.e., segment 0) and the last six elements (i.e., segment 1) are for partitions p_0 and p_1 , respectively. We then perform a parallel scan on del_p_wgt to accumulate delta partition weights for each partition. Specifically, after applying the parallel scan, the j^{th} element in segment s stores the accumulated delta partition weight from the first to the j^{th} vertex moves for partition s (i.e., a sub-sequence from the first to the j^{th} vertex moves). This accumulation allows us to quickly access each partition's accumulated delta partition weight if we apply all vertex moves in a sub-sequence of vertex moves. We then use these accumulated results to find the longest sub-sequence of vertex moves in the next step.

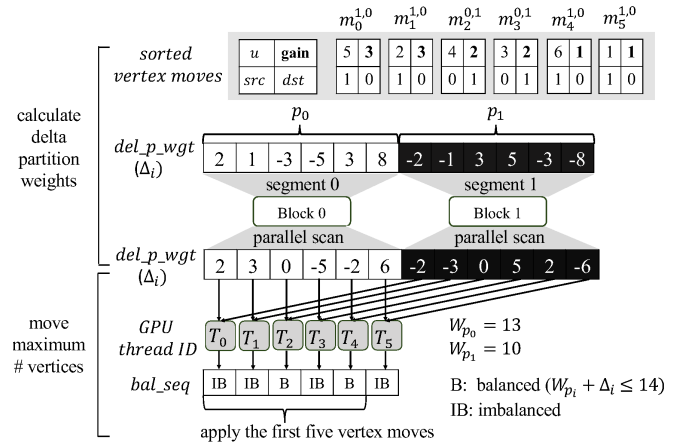


Figure 3: Illustration of the process to construct del_p_wgt and bal_seq with $k = 2$ under six vertex moves. Assuming current partition weights are 13 and 10 for p_0 and p_1 , respectively, with a balance constraint of 14.

Algorithm 3 presents the calculation of delta partition weights. We first sort vertex moves in the move buffer by gain in descending order in parallel using a parallel sorting algorithm (line 1). We assign each vertex move, $m_u^{src,dst}$, to a GPU thread, T_i based on its gid . Each GPU thread first gets the index of a vertex move's source (src_p_idx) and destination partition (des_p_idx) in del_p_wgt (lines 6-7). Each GPU thread then writes the corresponding delta partition weights to del_p_wgt (lines 8-9).

Finally, we apply our parallel scan kernel on each segment to obtain the accumulated delta partition weights per partition (lines 13-18). We launch our parallel scan kernel with the number of GPU blocks equal to k (i.e., number of partitions), where each GPU block conducts a parallel scan simultaneously for its assigned segment (line 15). To further improve performance, we utilize a CUDA warp-level primitive, `__shfl_up_sync`, for our parallel scan kernel.

Algorithm 3 Calculate delta partition weights

```

1: parallel sort the move buffer in descending order by gain
2:  $seg\_size \leftarrow \min(\#vertex\_moves, 1024)$ 
3:  $gid \leftarrow$  thread's global ID
4: /*assign a vertex move  $m_u^{src,dst}$  to a GPU thread  $T_i$  based on its  $gid$ */
5: parallel for each thread {
6:    $src\_p\_idx \leftarrow m_u^{src,dst}.src \times seg\_size + gid$ 
7:    $dst\_p\_idx \leftarrow m_u^{src,dst}.dst \times seg\_size + gid$ 
8:    $del\_p\_wgt[src\_p\_idx] \leftarrow -W_u$ 
9:    $del\_p\_wgt[dst\_p\_idx] \leftarrow W_u$ 
10:  return
11: }
12: /*assign segment  $seg_i$  of  $del\_p\_wgt$  to a GPU block  $b_i$ */
13: parallel for each block {
14:    $seg_i\_start \leftarrow b_i.ID \times seg\_size$ 
15:    $seg_i\_end \leftarrow seg_i\_start + seg\_size$ 
16:   parallel scan on  $seg_i$  /* __shfl_up_sync */
17:   return
18: }
```

3.2.3 Select vertex moves. In this step, we select the longest sub-sequence of vertex moves while ensuring that applying those vertex moves satisfies the balance constraint. This selection is based on our accumulated delta partition weights.

As shown in Figure 3, we maintain a bal_seq array to record the balanced condition for a sub-sequence of vertex moves. The value stored at index j in bal_seq indicates whether applying the sub-sequence of vertex moves from the first to the j^{th} results in a balanced partition. We then select the longest sub-sequence of vertex moves by finding the largest index j such that $bal_seq[j] = B$ (balanced). Finally, we apply all vertex moves in the longest sub-sequence of vertex moves.

In the example shown in Figure 3, each GPU thread checks if a sub-sequence of vertex moves results in a balanced partition and writes the result to the bal_seq array. Specifically, the first thread (T_0) checks the balanced result for the sub-sequence of vertex moves of the first vertex move, the second thread (T_1) checks for the sub-sequence of vertex moves from the first to the second vertex moves, and so on. Each thread fetches the accumulated delta partition weight for each partition from each segment in del_p_wgt , and checks whether every partition's current weight plus its accumulated delta partition weight satisfies the balance constraint. For example, assuming the balance constraint is 14, T_0 fetches $del_p_wgt[0]$ and $del_p_wgt[6]$ for p_0 and p_1 , and checks if both $W_{p_0} + \Delta_0 \leq 14$ and $W_{p_1} + \Delta_6 \leq 14$. If one of the partitions does not satisfy the balance constraint, the thread writes 'IB' (imbalanced); otherwise, 'B' (balanced) to its corresponding index in bal_seq .

After each thread finalizes bal_seq , we can observe that applying only the first vertex move results in an imbalanced partition ($bal_seq[0] = IB$). However, applying the first five vertex moves

helps to restore the partition result back to balance ($bal_seq[4] = B$). In the example shown in Figure 3, the longest sub-sequence is from the first to the fifth vertex moves. Since the sequence of vertex moves is sorted by gain in descending order, we can prioritize those vertex moves that make a substantial contribution to the overall improvement in cut size. Finally, we apply all vertex moves in the longest sub-sequence of vertex moves in parallel.

4 Experimental Evaluation

We evaluate the performance of G-kway on six industrial circuit graphs generated by [3, 4], where regular graphs are used to represent timing graphs. Additionally, we test G-kway's performance on four large non-circuit graphs (ldoor, NLR, delaunay, asia.osm) from DIMACS Graph Partitioning Challenge to demonstrate our applicability beyond CAD algorithms. Table 1 lists the statistics of each graph. We implement G-kway using C++17 and CUDA 12.0 and compile it with `nvcc` on a host compiler of GCC-8 with `-O3` enabled. We run experiments on a 64-bit Linux machine with 40 Intel Xeon Gold 6138 CPU cores at 2.00 GHz and 256 GB RAM. Our GPU is A6000 with 48 GB global memory.

4.1 Baselines

We consider `mt-metis v0.7.2` [7] and `GKSG` [2] as baseline partitioners. `Mt-metis` is a state-of-the-art CPU-parallel graph partitioner that renovates the sequential `Metis` algorithm [5] to a parallel target using `OpenMP`. `GKSG` is a state-of-the-art GPU-accelerated graph partitioner. Since `GKSG` is not open-source, we implemented its algorithm on our GPU except for the initial partitioning. Because the coarsest graph is typically very small, we do not observe any advantage in using GPU. In all experiments, we set the imbalance ratio to 3% and the coarsening threshold to $\frac{|V|}{20 \times (\log_2(k))}$. These settings are the same as the default values of `mt-metis` [7] and `GKSG` [2] that can produce the best results. All data is an average of ten runs.

4.2 Overall Performance Comparison

Table 1 compares the overall runtime and cut size results among G-kway, `GKSG`, and `mt-metis` at $k = 2$. We run `mt-metis` using 32 threads to achieve the best performance on our machine. In terms of runtime, G-kway outperforms `GKSG` and `mt-metis` across all graphs, with an average speedup of 3.8 $\times$ and 8.6 $\times$, respectively. The largest speedups we observe are 9.1 $\times$ over `GKSG` in `asia.osm` and 14.3 $\times$ over `mt-metis` in `wb_dma`. The significant improvement on runtime demonstrates the promise of our union find-based coarsening and independent set-based refinement algorithms. For the smallest graph, `ldoor`, G-kway still achieves 6.5 $\times$ and 1.6 $\times$ over `GKSG` and `mt-metis`. We attribute this significant speedup to our efficient coarsening algorithm that efficiently coarsen many vertices per subset, thus largely reducing the number of coarsening levels. Regarding cut size, G-kway outperforms `mt-metis` and `GKSG` on nearly all graphs. For instance, on `vga_lcd`, our cut size is 3.6 $\times$ better than `mt-metis`. We attribute this improvement to our coarsening algorithm, which results in better-coarsened graphs. Similar improvements can be found when comparing G-kway with `GKSG`.

4.3 Runtime Analysis

Figure 4 shows the speedup of G-kway over `mt-metis` (32 threads) and `GKSG` with different k on two circuit graphs (`wb_dma`, `tv80`) and two non-circuit graphs (`delaunay`, `ldoor`). Regardless of k , G-kway

Benchmark			GKSG		mt-metis		G-kway		Speedup vs		Cut size improvement vs	
Name	# Vertices	# Edges	Time (s)	Cut size	Time (s)	Cut size	Time (s)	Cut size	GKSG	mt-metis	GKSG	mt-metis
pci_bridge	12,394,539	15,809,551	0.89	5,114	3.21	4,773	0.27	4,293	3.5×	12.5×	1.2	1.1
vga_lcd	1,392,3210	24,904,499	1.33	5,661	3.80	17,237	0.46	4,737	2.9×	8.4×	1.2	3.6
wb_dma	19,686,000	20,236,297	1.21	7,131	3.81	7,844	0.32	6,915	4.6×	14.3×	1.0	1.1
usb	25,215,939	31,630,268	2.03	7,933	6.97	10,283	0.58	6,709	3.5×	12.0×	1.2	1.5
tv80	13,102,222	17,759,671	0.70	2,575	2.46	3,094	0.26	2,457	2.6×	9.3×	1.0	1.3
mem_ctrl	6,422,461	8,455,835	0.62	7,368	1.35	7,126	0.26	6,976	2.4×	5.3×	1.1	1.0
ldoor	952,203	2,278,5136	0.84	25,676	0.21	25,088	0.13	25,578	6.5×	1.6×	1.0	1.0
NLR	4,163,763	12,487,976	0.16	4,432	0.34	4,262	0.15	4,705	1.1×	2.3×	0.9	0.9
delaunay	16,777,216	50,331,601	0.48	12,650	2.23	8,614	0.27	8,463	1.8×	8.3×	1.5	1.0
asia.osm	11,950,757	12,711,603	0.96	9	1.30	8	0.11	7	9.1×	12.4×	1.3	1.1
Average									3.8×	8.6×	1.1	1.4

Table 1: Overall comparison of runtime (second) and cut size among GKSG, mt-metis (32 threads), and G-kway at $k = 2$. The last four columns represent the speedup and cut size improvement of G-kway over GKSG and mt-metis, respectively.

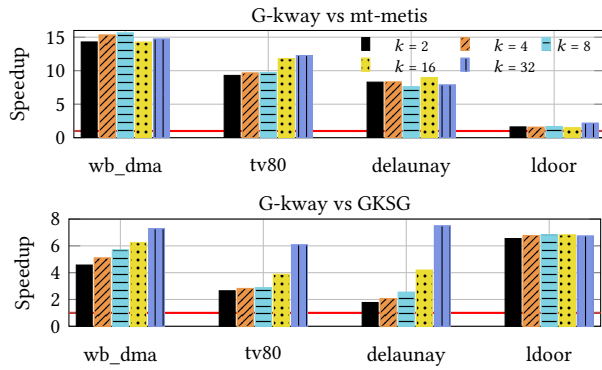


Figure 4: The speedup of G-kway over mt-metis (top) and GKSG (bottom) at different k .

is always faster than mt-metis and GKSG. Compared to mt-metis, G-kway achieves over 6× and 10× for more than 80% and 40% of the partitioning problem instances, respectively. For large graphs, such as wb_dma, our speedups are remarkable. The proposed GPU-accelerated coarsening and refinement algorithms bring significant performance benefits to parallel graph partitioning. Similar speedup values can also be observed in the comparison with GKSG. For instance, G-kway is 7× faster than GKSG on the wb_dma with $k = 32$.

4.4 Cut Size Analysis

Figure 5 shows the cut size improvement ratio of G-kway over mt-metis and GKSG at $k = \{2, 4, 8, 16, 32\}$. In general, G-kway can produce partitions with comparable quality to mt-metis and GKSG. Compared to GKSG, G-kway finds partitions with significantly less cut size for delaunay. We attribute this to our refinement algorithm. GKSG can only move a few vertices (e.g., eight) at one refinement iteration due to the memory limitation of its exponential enumeration algorithm. On the other hand, our refinement algorithm identifies a sequence of vertices through independent set finding and identifies the longest sub-sequence that satisfies the balance constraint. This approach allows G-kway to discover more valid moves in one iteration that can lead to a better cut size. However, moving too many vertices simultaneously can sometimes trap us in a local minima that produces a worse cut size than GKSG, such as tv80 at $k = 32$. Compared to other graphs, tv80 has longer path connectivity among

vertices which can benefit from more fine-grained refinement as GKSG.

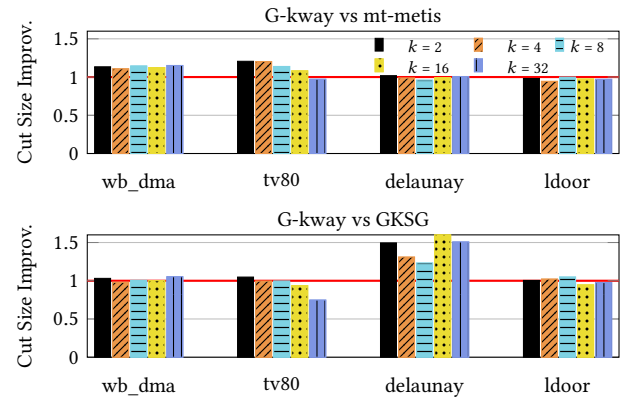


Figure 5: The cut size improvement ratio of G-kway over mt-metis (top) and GKSG (bottom) at different k .

5 Conclusion

In this paper, we have introduced G-kway, an efficient GPU-accelerated multilevel k -way graph partitioner. Experimental results have shown that G-kway outperforms the state-of-the-art CPU-based and GPU-based parallel partitioners.

Acknowledgment

This project is supported by NSF grants 2235276, 2349144, 2349143, 2349582, and 2349141.

References

- [1] Bahareh Goodarzi, Martin Burtcher, and Dhrubajyoti Goswami. 2016. Parallel graph partitioning on a CPU-GPU architecture. In *IPDPSW*. IEEE.
- [2] Bahareh Goodarzi, Farzad Khorasani, Vivek Sarkar, and Dhrubajyoti Goswami. 2019. High performance multilevel graph partitioning on GPU. In *HPDC*. IEEE.
- [3] Tsung-Wei Huang, Dian-Lun Lin, Chun-Xun Lin, and Yibo Lin. 2022. Taskflow: A Lightweight Parallel and Heterogeneous Task Graph Computing System. *IEEE TPDS* 33, 6 (2022), 1303–1320.
- [4] Tsung-Wei Huang and Martin DF Wong. 2015. OpenTimer: A high-performance timing analysis tool. In *ICCAD*. IEEE.
- [5] George Karypis and Vipin Kumar. 1998. A fast and high quality multilevel scheme for partitioning irregular graphs. *SISC* 20, 1.
- [6] Farzad Khorasani, Rajiv Gupta, and Laxmi N Bhuyan. 2015. Scalable simd-efficient graph processing on gpus. In *PACT*. IEEE.
- [7] Dominique LaSalle and George Karypis. 2013. Multi-threaded graph partitioning. In *IPDPS*. IEEE.